



INITIATION à la P.O.O en C#

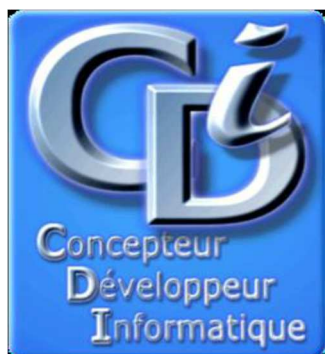


TABLE DES MATIERES

Table des matières	2
1. Préambule	3
2. Penser en termes d'objet	4
2.1 Présentation des notions d'encapsulation, de classe et d'objet.....	4
2.2 Présentation des notions d'héritage et de hiérarchie de classes.....	6
2.3 Réutilisation des objets	8
3. Attributs et méthodes	9
3.1 Variables d'instance.....	9
3.2 Variables de classe	10
3.3 Méthodes	11
3.4 Méthodes de classe et d'instances.....	12
3.5 Constructeurs	12
3.6 La surcharge de méthodes	13
3.7 Instanciation des objets	14
3.8 Notion de référence d'un objet.....	15
3.9 Le Polymorphisme	17
4. Echanges de messages entre classes.....	18
5. L'Agrégation et la Composition	18
6. Pour aller plus loin : Modélisation et pertinence du modèle.....	20
6.1 Exemple de hiérarchie de classes.....	21
6.2 Les classes abstraites.....	22
7. Pour aller plus loin : les Interfaces	22
8. Pour conclure sur l'orienté objet	23

1. PREAMBULE

Il est très important, pour écrire de façon efficace des programmes C# ou Java, de bien comprendre la philosophie du langage. Nous essaierons de rendre les choses aussi claires que possible, et utiliserons pour cela de nombreuses comparaisons.

Il existe plusieurs moyens de conceptualiser un programme informatique. L'approche classique consiste à considérer un programme comme une série d'instructions exécutées séquentiellement, ce qu'on a l'habitude d'appeler "**programmation procédurale**".

C# ou Java sont des langages "**orientés objet**". C'est là plus qu'un concept à la mode. C'est une véritable philosophie de la programmation. La programmation orientée objet est une façon d'élaborer des programmes informatiques qui tend à refléter la manière dont les objets à gérer sont assemblés et en communication dans le monde réel. Ainsi, une voiture est un ensemble complexe de composants en interactions les uns avec les autres mais elle forme un tout cohérent, capable d'agir (démarrer, freiner...), et doté de caractéristiques propres (sa couleur, son numéro d'immatriculation, sa vitesse...).

Pour parvenir à ce type de représentation du monde réel, la logique de programmation orientée objet s'appuie sur quelques principes qu'il s'agit ici de découvrir, avant de pouvoir les mettre en œuvre. Bien entendu, comme toute technologie, la programmation orientée objet introduit un nouveau vocabulaire, correspondant aux concepts sous-jacents, auquel il faudra bien se familiariser...

Nous aborderons les sujets suivants :

- l'organisation des programmes en éléments appelés **classes** et l'utilisation de ces classes pour définir des **objets** manipulables par programmation,
- la définition d'une classe sous les deux aspects de sa **structure**: le **comportement** qu'elle doit avoir et ses attributs, ses **données**,
- les relations entre les classes comme **l'héritage**, qui permet à une classe bénéficier des fonctionnalités d'une autre classe,
- les différents types **d'association entre objets**.

2. PENSER EN TERMES D'OBJET

En programmation orientée objet, un programme informatique se conceptualise comme un ensemble d'objets collaborant ensemble pour réaliser une tâche ; ***pour pouvoir programmer orienté objet, il faut au moins deux programmes, l'un qui offre des services et l'autre qui les utilise.***

Chaque objet est une partie distincte du programme, interagissant avec les autres parties selon des modes spécifiques très précisément décrits et contrôlés.

2.1 PRESENTATION DES NOTIONS D'ENCAPSULATION, DE CLASSE ET D'OBJET

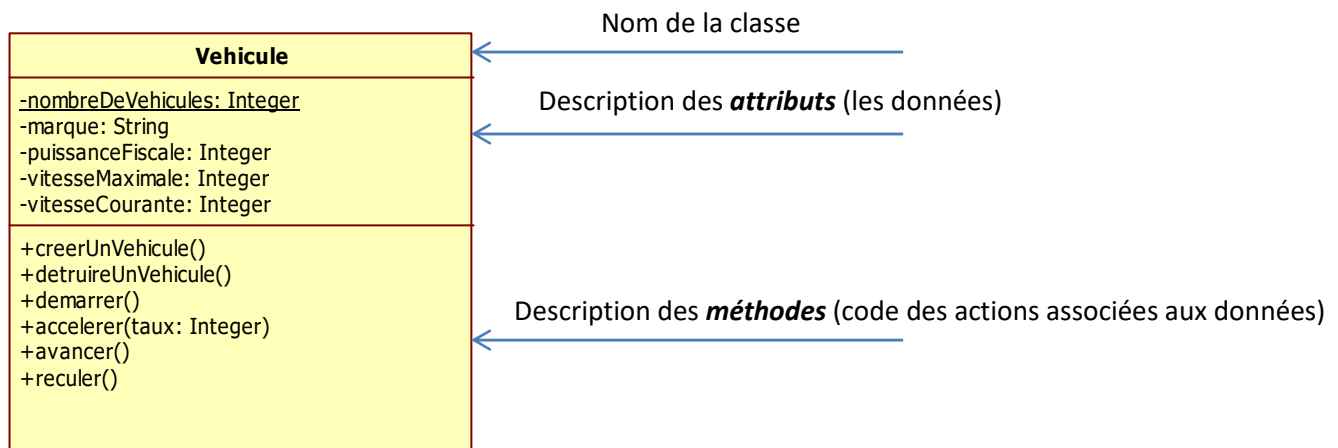
Un objet est un ensemble cohérent rassemblant des données et du code travaillant sur ces données. Le premier principe de l'orienté objet, ***l'encapsulation***, se rapporte à cet ***ensemble indissociable de données et traitements sur les données.***

Une classe peut être considérée comme un moule à partir duquel on peut créer des objets, chaque objet représentant un objet du monde réel à gérer par le programme. Ainsi, une classe `Vehicule` décrit les données et les actions que peut exécuter toute voiture et chaque voiture à gérer sera un exemplaire (une 'instance') de cette classe `Vehicule`.

Une classe décrit la structure interne d'un objet : les ***données*** qu'il regroupe, les ***actions*** qu'il est capable d'assurer sur ses données.

Le développeur écrit donc le code des classes, mais à l'exécution, ce sont les objets, instances des classes, qui réalisent les opérations.

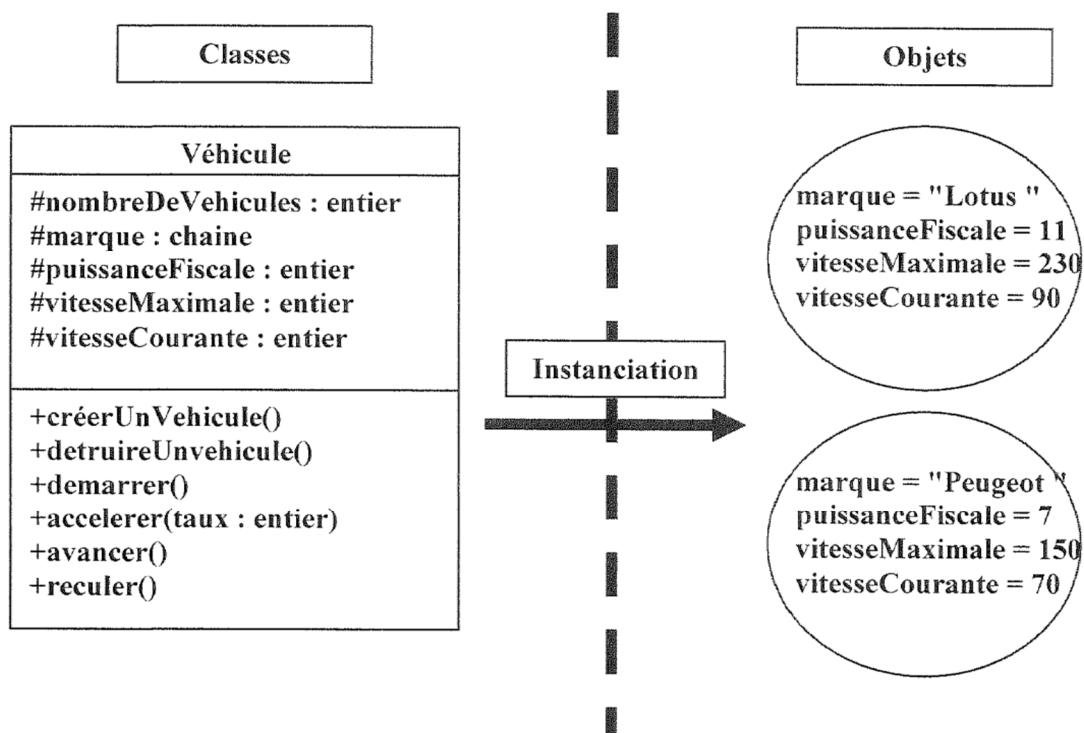
Considérons par exemple la modélisation d'un véhicule telle que présentée par la figure suivante qui représente une classe selon la notation UML :



Dans ce modèle, un véhicule est caractérisé par 4 données, ses ***attributs*** : une chaîne de caractères (sa `marque`) et trois entiers (`puissanceFiscale`, `vitesseMaximale` et `vitesseCourante`). Toutes ces données sont ***représentatives d'un véhicule particulier***, autrement dit, chaque objet véhicule aura sa propre copie de ses données : on parle alors ***d'attribut d'instance***.

En revanche, considérons l'attribut `nombreDeVehicules` chargé de compter le nombre de véhicules présents à un moment donné dans la classe. Il est incrémenté par l'opération `creerUnVehicule` et décrémenté par l'opération `destruireUnVehicule`. C'est un exemple typique **d'attribut partagé par l'ensemble des objets d'une même classe**. Il est donc nécessaire que tous les objets de cette classe partagent une **copie unique stockée au niveau de la classe**. On parle donc **d'attribut de classe**. Dans le code décrivant une classes, les attributs de classe sont repérés par un mot-clé (**static** en C# ou Java).

Si un programme doit manipuler deux véhicules, ilinstanciera deux objets depuis cette classe `Vehicule` ; **l'instanciation permet de passer de la classe (moule à objets) à l'exemplaire (objet)** ; on peut considérer que l'instanciation est l'opération qui 'donne vie' à l'objet ; en C# ou Java, on instancie un objet grâce au mot-clé **new**.



Le même raisonnement s'applique directement aux méthodes. En effet, de la même manière que nous avons établi une distinction entre attributs d'instance et attributs de classe, nous allons différencier **méthodes d'instances** et **méthodes de classe**.

Prenons par exemple la méthode `demarrer()`. Il est clair qu'elle s'applique individuellement à chaque véhicule. En outre, cette méthode va clairement utiliser ou modifier les attributs d'instance de l'objet auquel elle va s'appliquer, par exemple la `vitesseCourante` du véhicule en question. C'est donc une méthode d'instance.

Considérons désormais le cas de la méthode `creerUnVehicule()`. Son but est de créer un nouveau véhicule en réservant la place mémoire nécessaire ; cette création d'un objet est donc assurée par une méthode de classe. L'objet instancié aura, dans un second temps, le loisir de positionner des valeurs initiales dans chacun de ses attributs d'instance puis de les faire évoluer (l'initialisation des attributs d'un objet serait donc une méthode d'instance).

Nous constatons que la création/instanciation d'un nouvel objet est constituée de deux phases :

- une phase du ressort de la classe (et du système d'exploitation) : allouer de la mémoire pour le nouvel objet et lui fournir un contexte d'exécution,
- une phase du ressort de l'objet : initialiser ses attributs d'instance.

En langage C# ou Java ces opérations sont réalisées dans une **méthode spéciale**, ni vraiment méthode d'instance, ni méthode de classe : le **constructeur de la classe**, qui est **appelé automatiquement par l'opérateur new**.

Cette méthode particulière **porte le nom de la classe et accepte éventuellement en paramètres les valeurs initiales** pour l'objet à créer. Par exemple, pour la voiture 'Lotus', l'instanciation pourrait être réalisée de la manière suivante en C# ou Java :

```
... new vehicule("Lotus", 11, 230, 90) ;
```

La notion de constructeur est détaillée plus loin.

2.2 PRESENTATION DES NOTIONS D'HERITAGE ET DE HIERARCHIE DE CLASSES

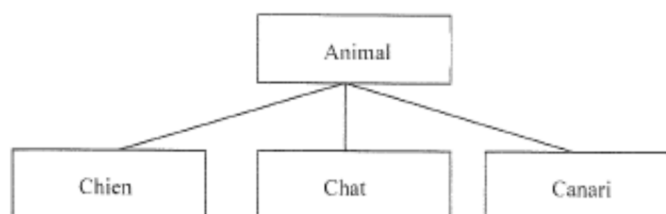
Le deuxième principe de l'orienté objet, l'**héritage**, est que **l'on peut définir des classes à partir d'autres classes, ce qui évite de tout réécrire** quand des classes ont des caractéristiques ou des actions en commun.

L'héritage est un concept majeur de la programmation orientée objet; c'est un mécanisme permettant à une classe d'hériter de (de pouvoir utiliser) toutes les méthodes et attributs d'une autre classe.

Prenons un exemple. Si notre problème concerne les animaux, nous pouvons créer une classe que nous appellerons **Animal**.

Si nous devons considérer les chiens, les chats et les canaris, nous créerons trois classes dérivées de la classe **Animal** : les classes **Chien**, **Chat** et **Canari**.

Peu importe que cette division ne soit pas pertinente dans l'univers réel. Il suffit qu'elle le soit dans celui du problème à traiter. Nous représenterons cette division de l'univers de la façon suivante :



Un chien est bien un animal particulier ; on dit que la classe **Chien** **hérite, étend ou dérive** de la classe **Animal** ; la classe **Animal** est une **généralisation** des classes **Chien**, **Chat** et **Canari**.

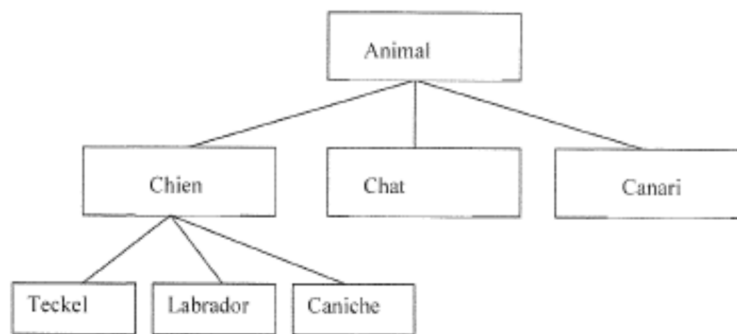
L'héritage dénotant une relation de généralisation / spécialisation, on peut traduire toute relation d'héritage par la phrase :

« La classe dérivée **est une** version spécialisée de sa classe de base ».

On parle également de **relation est-un** pour traduire le principe de généralisation / spécialisation. Ici, un chien est bien un animal, une sorte d'animal.

Bien entendu, chacune des classes symbolisées ci-dessus est dotée d'attributs et méthodes mais en y regardant de plus près, un animal, comme un chien ou un chat est capable de `seNourrir()`, `seDeplacer()` et est doté au moins d'un nom. Avec le mécanisme d'héritage, **ces attributs et méthodes communs seront décrits uniquement dans la classe `Animal` chacune des classes dérivées en héritant automatiquement**. Il ne reste à écrire dans les classes spécialisées que ce qui est réellement spécifique : `ronronner()` et `miauler()` pour un chat, `aboyer()` pour un chien, ou encore `siffler()` pour un canari, par exemple.

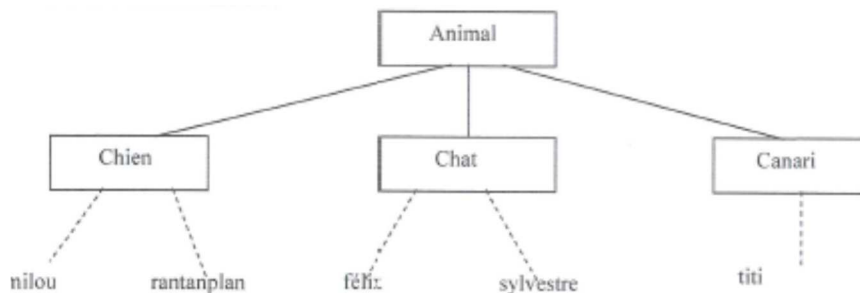
Nous pouvons, si nécessaire, créer encore de nouvelles classes dérivées, par exemple :



Il est d'usage de représenter ainsi la hiérarchie des classes, sous forme d'arbre inversé. La classe la plus générale se trouve à la racine (en haut, puisque l'arbre est inversé).

Notez que ces deux exemples sont incomplets. En effet, en Java ou C# toutes les classes dérivent d'une classe de base fournie, la classe **Object**.

Considérons maintenant le modèle suivant :



Il est évident que le rapport entre `milou` et `Chien` n'est pas de même nature que le rapport entre `Chien` et `Animal`.

`Chien` est une classe dérivée de `Animal`.

En revanche, `milou` n'est pas une classe. C'est une instance, un représentant de la classe `Chien`.

2.3 REUTILISATION DES OBJETS

Grâce à C# ou Java vous pouvez par exemple créer une classe pour représenter les "Boutons de commande".

Lors du développement de la classe `BoutonDeCommande`, on pourra définir pour elle les fonctions suivantes :

- le texte identifiant le rôle du bouton,
- la taille du bouton,
- des aspects spécifiques de son apparence (ombré, 3D ...).

Après avoir défini la classe `BoutonDeCommande`, vous pouvez créer des instances de ce bouton. En d'autres termes des **objets de type** `BoutonDeCommande`.

Ces objets adoptent toutes les caractéristiques de base d'un bouton à cliquer tel qu'il est défini par la classe, mais chacun peut avoir un aspect différent et des comportements légèrement distincts.

En créant tout d'abord la classe `BoutonDeCommande`, vous n'avez pas besoin de réécrire continuellement le code pour chaque bouton de commande à utiliser dans vos programmes.

Quand vous écrivez un programme C# ou Java, vous concevez et construisez un jeu de classes. Quand votre programme s'exécute, des objets sont instanciés à partir des classes puis utilisés et personnalisés par leurs valeurs d'attributs selon vos besoins.

Heureusement, vous n'avez pas besoin de tout construire à partir de zéro, le langage C# ou Java incluant des centaines de classes mettant en œuvre la plupart des fonctionnalités élémentaires dont vous avez besoin dans vos applications (comme des boutons de commande justement !). On parle de **Bibliothèques des classes** C# ou Java, ou encore de **Framework**.

Se servir du langage C# ou Java signifie utiliser ces bibliothèque de classes ainsi que les opérateurs et mots clés standard reconnus par le compilateur C# ou Java pour créer vos propres classes et ensuite en instancier et manipuler des objets.

3. ATTRIBUTS ET METHODES

Une classe C# ou Java est constituée en deux types d'informations distinctes :

- les **attributs**,
- les **méthodes**.

Les attributs sont les données qui différencient un objet d'un autre. Ils peuvent servir à déterminer l'aspect, l'état et d'autres qualités d'un objet appartenant à une classe.

Dans une classe les attributs sont définis par des variables. Ces "variables" pourront prendre une valeur différente pour chaque objet et évoluer dans le temps.

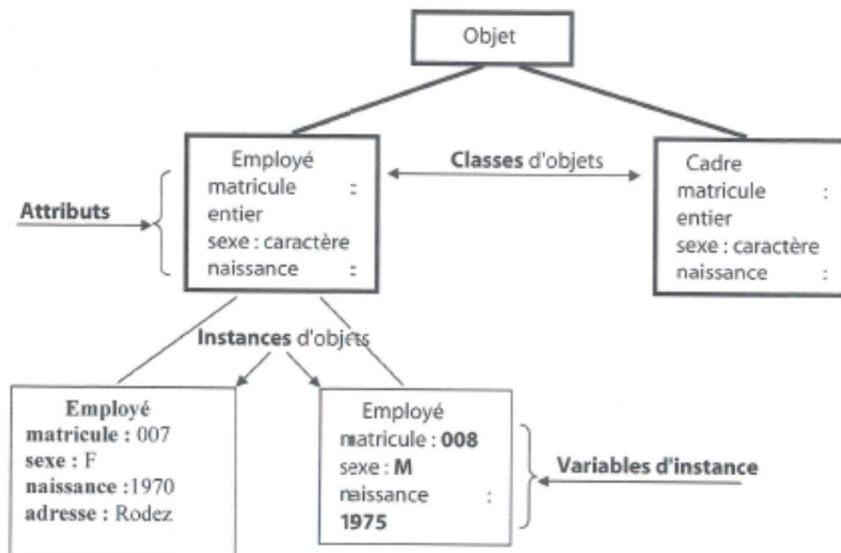
Les méthodes, qui définissent le comportement des objets, désignent la façon dont les objets d'une classe peuvent effectuer une action sur eux-mêmes ou éventuellement sur d'autres objets.

3.1 VARIABLES D'INSTANCE

Une **variable d'instance** définit un attribut d'un objet particulier.

La classe de cet objet définit de quelle sorte d'attribut il s'agit, de quel **type de donnée**. Chaque instance stocke sa propre valeur pour cet attribut.

Exemple : Gestion du personnel



Chacun des 2 objets, instances de la classe `Employé`, possède ses propres valeurs d'attributs.

3.2 VARIABLES DE CLASSE

Pour certaines données, il est plus intéressant de gérer une valeur commune à tous les objets de la classe. C'était le cas de l'attribut `nombreDeVehicules` de la classe `Vehicule` dans le premier exemple. Ces attributs sont représentés par des **variables de classe**.

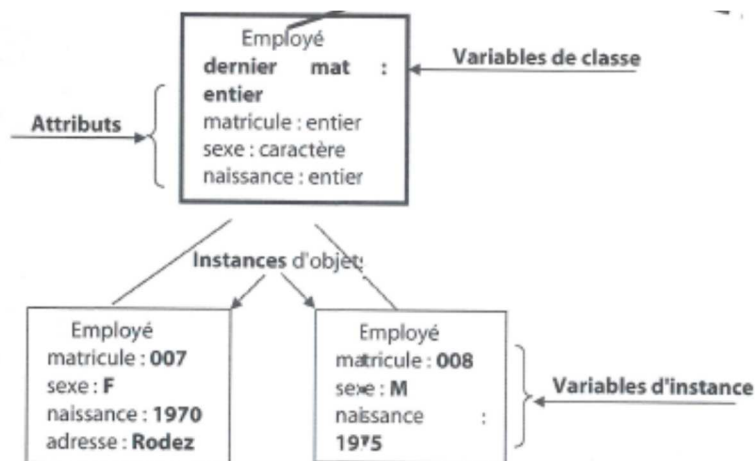
Une variable de classe définit un attribut d'une classe entière. **Cette variable s'applique à toutes les instances de la classe, une seule valeur est stockée quel que soit le nombre d'objets créés.**

Ce type de variable est dit *statique*, le mot-clé correspondant en C# ou Java est **static**.

Exemple : Gestion du personnel

Dans le cas de notre classe `Employé`, nous supposons que nous utiliserons comme numéro de matricule le numéro d'ordre de l'employé. Le premier aura le numéro 1, le second le numéro 2, etc. Le problème qui se pose est alors de garder en mémoire le numéro du dernier employé créé.

La solution consiste à ajouter une variable *statique* (de classe). Notre classe `Employe` devient :



Dès lors, la classe `Employe` elle-même sera capable de fournir le prochain numéro de matricule à attribuer au nouveau salarié ; cette opération concerne bien uniquement les salariés mais n'est pas spécifique à un salarié en particulier ; il est donc logique que cela soit géré et mémorisé par la classe `Employe` elle-même et non par les instances de cette classe.

3.3 METHODES

Le **comportement** désigne la façon dont les objets d'une classe peuvent effectuer une action sur eux-mêmes ou sur d'autres objets. Le comportement peut servir à **modifier les attributs d'un objet**, à **recevoir des informations provenant d'autres objets** et à **envoyer des messages à d'autres objets** pour leur demander d'accomplir des tâches. Le comportement d'une classe d'objets est **mis en œuvre à l'aide des méthodes ; il est défini par l'ensemble des méthodes disponibles dans la classe**.

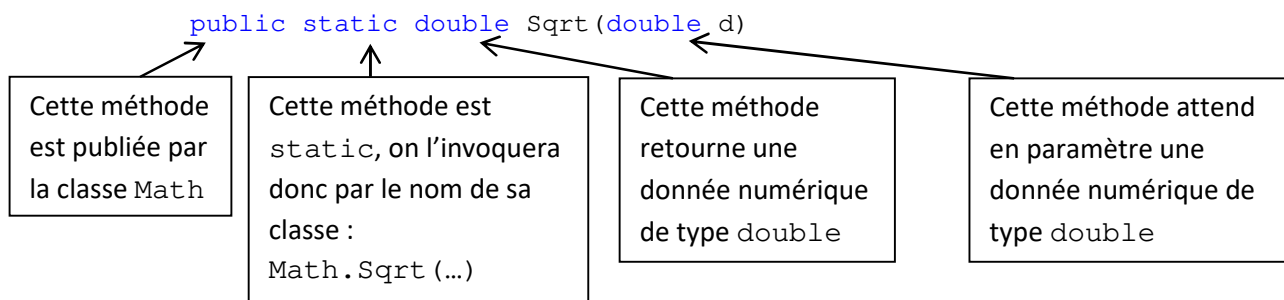
Les objets, instances de classes, communiqueront entre eux grâce aux méthodes. Un objet peut appeler (déclencher, 'invoker') des méthodes situées dans un autre objet pour diverses raisons :

- signaler une modification apportée sur ses propres données,
- demander à l'autre objet de changer quelque chose sur lui-même,
- demander à un autre objet de faire quelque chose...

En pratique, une méthode est constituée d'un groupe d'instructions capable d'assurer une action élémentaire sur les données de l'objet ou de communiquer avec d'autres objets (une méthode peut être considérée comme un *sous-programme*, une *procédure* ou une *fonction* dans un langage de programmation procédural).

Une méthode est **identifiée par un nom**, une **liste éventuelle de paramètres** et une **valeur de retour** ; ces trois éléments forment la **signature de la méthode**.

Ainsi, la méthode qui calcule une racine carrée se nomme classiquement `Sqrt()`, attend un paramètre numérique (le nombre dont il faut calculer la racine), et retourne une valeur numérique (la racine calculée). En C#, la signature de la méthode fournie par la classe `Math` est donc :



Note : Les méthodes sont faciles à identifier car **leur nom est toujours suivi de parenthèses encadrant les éventuels paramètres**. Si une méthode ne prend pas de paramètres, les parenthèses sont vides mais doivent être présentes.

Par exemple, une classe `Employe` pourrait avoir une méthode `AfficherMatricule()` qui se contente d'afficher la valeur de la variable `matricule`.

Elle pourrait avoir aussi une méthode `AfficherAge()` qui, elle, devra effectuer un certain nombre d'opérations pour pouvoir mener à bien sa tâche : par exemple interroger un objet pour connaître la date du jour, effectuer une soustraction (date du jour – date de naissance) puis une série de conversion afin d'afficher l'âge.

3.4 METHODES DE CLASSE ET D'INSTANCES

De même qu'il existe des variables d'instance et de classe, ***il existe des méthodes d'instance et des méthodes de classe.***

Ainsi la méthode précédente `Math.Sqrt()` est une méthode de classe mais la méthode `AfficherAge()` est bien une méthode d'instance.

Les méthodes d'instance, sont tellement répandues qu'on les appelle simplement méthodes.

3.5 CONSTRUCTEURS

Revenons sur la notion de constructeur abordée plus haut.

Un constructeur est une **méthode** qui a pour particularité de posséder le **même nom que la classe** à laquelle il appartient. Il est facile de distinguer un constructeur du nom de la classe à laquelle il appartient, puisqu'il est suivi de parenthèses encadrant ses éventuels arguments, comme toute méthode. De plus, ***un constructeur n'a pas de valeur de retour.***

Voyons comment pourrait se présenter un constructeur pour notre classe `Employe`. Pour créer un employé, il faut lui attribuer un matricule. Pour cela, il suffira d'interroger la classe pour connaître le dernier numéro d'employé, d'augmenter ce nombre d'une unité et d'utiliser le résultat comme matricule du nouvel employé.

Voici ce que cela pourrait donner en langage C# :

```
// classe Employe - dérive de la classe Object
public class Employe : Object
{
    // attributs
    private int matricule; // attribut d'instance
    private static int dernierMat; // attribut de classe

    // constructeur
    public Employe()
    {
        matricule = Employe.prochainMatricule() ;
    }

    // méthode de classe
    private static int prochainMatricule()
    {
        return ++dernierMat;
    }
    // méthode d'instance
    public void AfficherMatricule()
    {
        System.Console.WriteLine(matricule);
    }
}
```

Nous avons ici écrit explicitement que la classe `Employe` étend (ou 'dérive de') la classe `Object`, ce qui n'était pas nécessaire, puisque nous avons vu précédemment que toutes les classes *étendent* par défaut la classe `Object`.

Ensuite, nous déclarons notre premier attribut stocké dans une variable d'instance. Ce sera un nombre entier (`int`) et son accès sera limité à cette classe (`private`). Le nom que nous lui donnons est `matricule`.

```
private int matricule;
```

La ligne suivante fait exactement la même chose en déclarant une variable entière de classe appelée `dernierMat`.

```
private static int dernierMat;
```

Comme nous avons employé le mot `static`, il n'existera qu'un seul exemplaire de cette variable, qui appartiendra à la classe `Employe` et non à ses différentes instances.

On trouve ensuite la définition de la méthode constructeur de la classe `Employe` et la méthode de classe `prochainMatricule()`. Pour chaque méthode, la première ligne consiste en une déclaration par la *signature de la méthode*. Cette déclaration comporte le *nom* de la méthode, le *type de la valeur de retour* et les *paramètres* qu'elle emploie. La déclaration de la méthode se termine par une accolade ouvrante qui ouvre un bloc. Ce bloc contient la *définition* de la méthode. Il se termine par une accolade fermante. Ici la définition du constructeur ne comporte qu'une seule ligne qui fait appel à un service interne, une autre méthode de la classe `Employe` :

```
matricule = Employe.prochainMatricule() ;
```

La signification (simplifiée) de la méthode appelée est : "Augmenter d'une unité la valeur de `dernierMat` et retourner cette valeur."

Il faut bien comprendre que cette simple ligne de la méthode `prochainMatricule()` fait deux choses différentes :

- elle *augmente et modifie* la valeur de `dernierMat`,
- elle *retourne la valeur* de `dernierMat` à la méthode qui l'a appelée, ici, le constructeur de la classe, qui s'empresse de l'affecter au `matricule` du nouvel employé.

Nous avons ensuite une méthode d'instance qui affiche le matricule.

Cette méthode ne retourne aucune valeur. Il existe pour cela, un type particulier : `void`, qui signifie « rien ».

3.6 LA SURCHARGE DE METHODES

La surcharge de méthode est un mécanisme fréquemment proposé par les langages de programmation orientés objet et qui ***permet de réaliser des variantes d'une même méthode en associant au même nom de méthode différentes signatures.***

Par exemple, on pourrait définir plusieurs surcharges de la méthode constructeur de la classe `Employe` :

- la forme actuelle `public Employe() {...}` sans paramètre,
- une version acceptant en paramètre un numéro de matricule quand il est déjà connu de l'utilisateur `public Employe(int unMatricule) {...}`

Ainsi, le .Net Framework propose une méthode `MessageBox.Show()` proposant plus de 20 surcharges ce qui permet au programmeur de personnaliser complètement les boîtes d'alerte.

3.7 INSTANCIATION DES OBJETS

Voyons maintenant ce qui se passe lorsque nous créons un objet, c'est-à-dire une instance d'une classe.

Nous avons vu que la création d'une instance est réalisée par un autre programme (*en objet, il faut être au moins 2 !*) et met en jeu l'exécution du constructeur de sa classe. Précisons maintenant la syntaxe à utiliser pour déclencher cette création. Pour créer un objet en C# ou Java, il faut utiliser le mot clé **new**, suivi du nom du constructeur de la classe, avec les parenthèses contenant les paramètres éventuels.

Par exemple, pour créer une instance de la classe **Employe** (ou, en d'autres termes un objet de type **Employe**), nous utiliserons la syntaxe suivante :

```
new Employe();
```

Les questions qui se posent immédiatement sont : que devient l'objet que nous avons créé, et comment pouvons-nous l'utiliser ? Pour répondre à ces questions, nous pouvons tester le programme suivant :

```
public class Test
{
    // programme de démarrage
    public static void Main(String[] args)
    {
        new Employe();
    }
}
```

```
public class Employe
{
    int matricule;
    static int dernierMat;

    public Employe()
    {
        ...
    }
}
```

Nous avons ajouté une classe **Test** contenant une méthode nommée **Main()** de façon à pouvoir démarrer l'exécution de notre programme.

Que fait ce programme ? Pour la partie qui nous intéresse, il crée une instance de **Employe**, ce qui est réalisé par la ligne :

```
new Employe();
```

Lorsque cette ligne est exécutée, le programme **charge la classe** `Employe`, vérifie si la classe `Employe` possède un constructeur correspondant, dans notre cas, un constructeur sans paramètres. Comme c'est bien le cas, **ce constructeur est exécuté**.

Une instance d'`Employe` est donc créée dans la mémoire de l'ordinateur. Que pouvons-nous faire de cette instance ?

Dans l'état actuel des choses, rien.

Lorsque vous créez un objet, vous ne savez pas du tout où le système le place en mémoire. *Tant que vous le tenez, vous pouvez l'utiliser*. Si vous le lâchez, le système ne vous le rendra que si vous êtes capable de lui indiquer de façon non équivoque de quel objet il s'agit.

Vous ne pouvez pas dire : "Je voudrais afficher le numéro matricule de l'employé que j'ai créé il y a cinq minutes à peine."

En revanche, vous pouvez le faire au moment de la création, lorsque vous 'tenez' en quelque sorte l'objet en main.

Modifions le programme **Test** de la façon suivante :

```
public class Test2
{
    public static void Main(String[] args)
    {
        new Employe().AfficherMatricule();
    }
}

public class Employe
{
    ...
}
```

Cette fois, on devrait obtenir l'affichage du matricule de l'employé créé par le programme principal. C'est plutôt rassurant. Nous voilà maintenant certains qu'un objet, instance de la classe `Employe`, a bien été créé.

Sommes-nous, pour autant, plus avancés ? Pas vraiment. En effet, dès que l'exécution de la ligne :

```
new Employe().AfficherMatricule();
```

est terminée, l'objet n'est plus accessible. Il se trouve toujours dans la mémoire, mais vous l'avez 'lâché' et n'avez plus aucun moyen de le 'rattraper'. Il continue cependant d'exister un certain temps en mémoire avant d'être éliminé.

L'environnement d'exécution C# ou Java inclut une tâche de fond appelée **garbage collector** (ce qui signifie littéralement "ramasseur d'ordures") qui assure en permanence la libération des espaces mémoire occupés par des objets devenus inutiles, c'est-à-dire des objets qui ne sont plus connus d'aucun programme en cours d'exécution. *Pour ne pas lâcher un objet, il faut en connaître la **référence**.*

3.8 NOTION DE REFERENCE D'UN OBJET

Nous pouvons utiliser immédiatement un objet créé, mais si nous voulons pouvoir l'utiliser plus tard, il faut s'assurer que nous pourrions le récupérer. Pour cela, il suffit de pouvoir l'identifier.

Dans le langage courant, il est fréquent d'utiliser des artifices pour identifier les objets. Les personnes, les animaux domestiques et certains objets (les bateaux, les villes, les pays, les fleuves, etc.) sont identifiés à l'aide d'un nom propre. D'autres objets sont identifiés par un moyen différent : numéro d'immatriculation, titre (pour une œuvre littéraire ou musicale), etc.

En programmation orientée objet, tous les objets sont logés à la même enseigne : ils sont identifiés à l'aide d'une **référence**.

Rien n'est plus simple que de créer une référence d'objet. Il suffit pour cela de déclarer une variable en indiquant vers quelle classe d'objets elle est susceptible de pointer.

Par exemple, pour créer la référence **josiane** pouvant pointer vers une instance d'`Employe`, nous écrivons

```
Employe josiane;
```

Vers quoi pointe cette référence ? Pour l'instant, vers rien. En effet, nous avons *déclaré* la référence, ce qui suffit à le faire exister. En revanche, nous ne l'avons pas *initialisée*, c'est-à-dire que nous ne lui avons pas attribué sa valeur initiale. Pour l'instant, elle ne pointe donc vers rien, sa valeur est `null`.

Nous avons vu qu'un objet pouvait être créé à l'aide de son constructeur et de l'opérateur **new**. Nous ne pouvons récupérer la référence d'un objet nouvellement créé qu'au moment de sa création/instanciation.

En effet, si nous écrivons les lignes :

```
Employe josiane;  
new Employe();
```

il n'y a ensuite aucun moyen d'établir un lien entre la référence créée à la première ligne et l'objet instancié à la seconde. Il faut donc effectuer l'affectation en même temps que la création, de la façon suivante :

```
Employe josiane;  
josiane = new Employe();
```

Ce qui ne signifie en aucun cas que la référence **josiane** soit égale à l'objet créé, mais simplement qu'il pointe vers cet objet. Cette variable **josiane** pourra donc servir à manipuler cet objet.

C# nous permet même de condenser ces deux lignes en une seule sous la forme :

```
Employe josiane = new Employe();
```

Modifions le programme précédent :

```
public class Test3  
{  
    public static void Main(String[] args)  
    {  
        Employe josiane = new Employe();  
        josiane.AfficherMatricule();  
        josiane.AfficherMatricule();  
    }  
}  
  
public class Employe  
{  
    ...  
}
```

A l'exécution, ce programme affiche deux fois le matricule du même objet instancié, ce qui aurait été tout à fait impossible sans l'utilisation de la référence.

Au moment de l'instanciation, l'opérateur **new** a créé l'objet et l'a mis dans une place disponible en mémoire. Cette adresse mémoire est conservée dans la variable **josiane**. On dit que **josiane** contient une référence vers l'objet.

Pour résumer, l'opérateur **new** réalise donc trois opérations :

- Il **charge en mémoire** un exemplaire de la classe (du moins, un espace, un contexte d'exécution, négocié avec le système d'exploitation qui, lui, gère physiquement la mémoire centrale),
- Il **récupère la référence** à cet espace mémoire de manière à pouvoir adresser l'objet par la suite,
- Il **déclenche l'exécution du constructeur** de la classe.

3.9 LE POLYMORPHISME

Le **polymorphisme** est le troisième des trois grands principes sur lequel repose l'orienté objet. **Le polymorphisme permet à une méthode d'adopter différents fonctionnements (plusieurs formes) pour des classes différentes dérivant d'une même classe de base.**

Nous allons démontrer la puissance du polymorphisme au travers de l'exemple des classes d'objets graphiques. Un document dessin peut être vu comme une collection d'objets graphiques de type `Cercle`, `Rectangle`, `Ligne`, ou toute autre sorte d'objet graphique qui pourrait dériver de la classe `ObjetGraphique`.

Si nous voulons dessiner un dessin tout entier, il nous faudra appeler la méthode `Dessiner()` pour chacun des objets constituant ce dessin. Hors, nous avons pris soin de conserver la même signature pour les différentes méthodes `Dessiner()` de tous les objets appartenant à la hiérarchie d'`ObjetGraphique`. **Chaque classe dérivée définit ou redéfinit à sa manière cette méthode** `Dessiner()` car on ne dessine pas un cercle comme un rectangle ou un triangle.

Le programme principal de dessin n'a plus qu'à appeler la méthode `Dessiner()` pour chacune des formes (chacun des objets) qui constituent le dessin ; le polymorphisme de la méthode `Dessiner()` garantit que la bonne méthode sera appelée pour chacun des objets.

4. ECHANGES DE MESSAGES ENTRE CLASSES

Dans l'exemple des salariés, le programme principal qui utilise la classe `Employe` n'a aucun élément d'information sur la mécanique interne mise en œuvre. Il ne sait pas si le traitement requis pour l'instanciation de l'objet a demandé l'intervention de plusieurs méthodes ou même la création d'objets temporaires.

Le **principe d'encapsulation** et les **modificateurs d'accès** `private/public` ont permis à la classe `Employe` **d'offrir des services tout en sécurisant ses propres données**.

Les services offerts par un objet ne sont déclenchables qu'au travers de messages reçus qui invoquent les méthodes publiées.

C'est bien ce que réalisaient les instructions :

```
josiane = new Employe(); // invoque le constructeur de la classe
et
josiane.AfficherMatricule(); // invoque une méthode d'instance
```

dans l'exemple précédent.

La liste des messages auxquels est capable de répondre un objet constitue son **interface** : c'est la partie publique d'un objet qui lui permet de communiquer avec d'autres objets. Tout ce qui concerne son implémentation doit rester caché au programme utilisateur : c'est la partie privée de l'objet ; elle doit pouvoir évoluer sans remettre en cause le service offert (par exemple remplacer un tableau de variables par une liste chaînée, ou changer de système de base de données).

Bien entendu, tous les objets d'une même classe présentent la même interface ; en revanche deux objets appartenant à des classes différentes peuvent présenter la même interface, c'est-à-dire offrir les mêmes types de services (par exemple, une classe émettant un message en WI-FI et une autre produisant un signal électrique pour câble Ethernet offrent les mêmes services aux classes qui construisent des messages à transmettre).

5. L'AGREGATION ET LA COMPOSITION

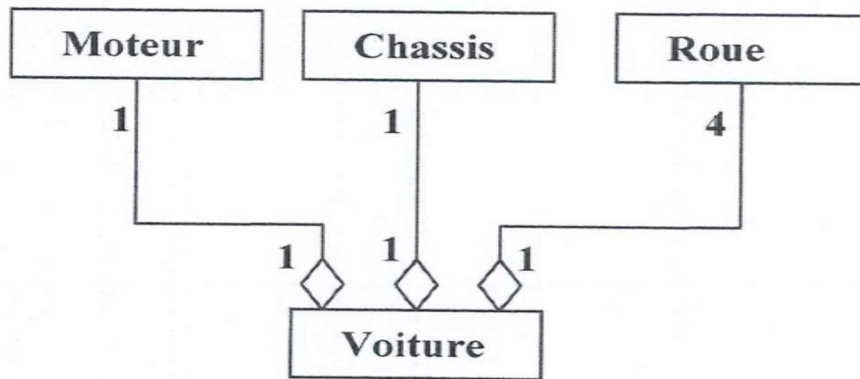
L'agrégation est un type de relation entre deux classes qui traduit cette fois les relations « Est composé de ... » ou « Possède ... » ou encore « a ... ».

Par exemple, dans un système mécanique, on pourrait considérer que la classe `Voiture` est composée d'une instance de la classe `Moteur`, quatre instances de la classe `Roue` et une instance de la classe `Chassis`. L'instanciation d'une `Voiture` induit nécessairement des instanciations d'objets et l'utilisation d'attributs qui sont eux-mêmes des références d'objets.

L'une des caractéristiques principale de l'agrégation est sa *cardinalité*.

Considérons par exemple l'agrégation de roues par une voiture. Une voiture possède exactement 4 roues (je laisse la roue de secours) et chaque roue ne peut pas être possédée par plus d'une voiture. La cardinalité de l'agrégation est donc de 1 côté agrégateur et de 4 du côté agrégé.

La figure suivante permet de fixer les idées :



La traduction en code orienté objet nécessiterait :

- 4 classes, **Voiture**, **Moteur**, **Chassis** et **Roue**
- 2 attributs dans la classe **Voiture**, respectivement de type **Moteur** et **Chassis** pour représenter les relations de cardinalité 1
- un attribut de type tableau de 4 références d'objets de type **Roue** dans la classe **Voiture**.

La composition est une relation d'agrégation particulière entre objets qui représente une relation forte entre composants et composé.

Pour une voiture, châssis, moteur et roues peuvent exister en tant que telles. Un livre est bien composé d'une couverture et de multiples feuilles mais chaque feuille n'a pas de sens individuellement ; la destruction d'un livre entraîne la destruction de toutes ses feuilles (ce qui n'est pas le cas des pièces détachées d'une voiture). En orienté objet, on parle d'agrégation dans le cas de la voiture et de composition dans le cas du livre.

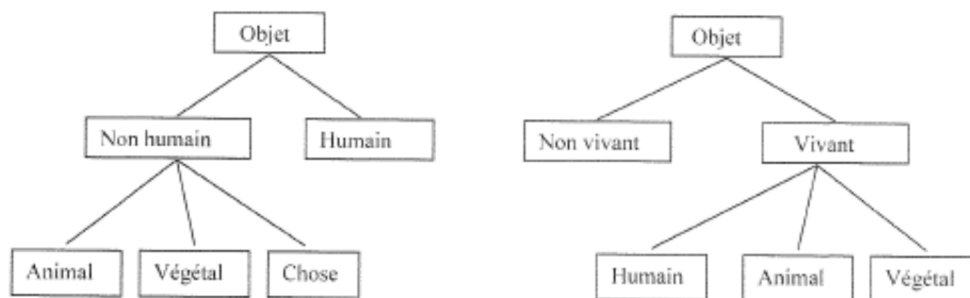
6. POUR ALLER PLUS LOIN : MODELISATION ET PERTINENCE DU MODELE

La structure que nous construisons ainsi représente l'univers du problème à traiter. S'il s'agit d'un problème de l'univers "réel", on peut être tenté d'essayer de construire un modèle du monde réel. C'est là un piège dans lequel il faut éviter de tomber, et ce pour deux raisons.

La première est qu'un modèle du monde réel prendra en compte de très nombreux critères non pertinents en ce qui concerne notre problème à résoudre. Ainsi, dans l'exemple des animaux il est inutile de décrire comment chaque animal se nourrit ou se déplace si l'application ne se préoccupe que des sons émis par les animaux.

La deuxième est que ce que nous croyons être la structure du monde réel est en réalité la structure de ce que nous en percevons et que cela dépend de très nombreux facteurs (culturels et psychologiques en particulier) qui sont le plus souvent totalement parasites en ce qui concerne les problèmes que nous avons à traiter.

Par exemple, selon que nous sommes plutôt portés vers le 'créationnisme' ou 'l'évolutionnisme', nous préférons probablement l'un ou l'autre des modèles suivants :



Peu importe lequel de ces modèles correspond le mieux au monde "réel".

La seule chose qui compte véritablement est de savoir lequel est le plus efficace pour la représentation de notre problème.

Une classe ne peut posséder qu'une *superclasse* mais avoir une infinité de sous-classes dérivées. Les sous-classes héritent de tous les attributs et de toutes les méthodes de la superclasse. En pratique cela signifie que si la superclasse possède des méthodes et des attributs dont votre classe a besoin, vous n'avez pas à réfléchir ou à copier ce code pour obtenir les mêmes méthodes ou attributs.

Votre classe hérite de toutes les fonctionnalités des classes supérieures en plus de ses caractéristiques propres.

Si vous créez un grand nombre de classes, il est logique que vos classes héritent de la hiérarchie de classes fournie dans le Framework et qu'elles constituent elles-mêmes une hiérarchie. Organiser ainsi vos classes demande une organisation soignée, mais les avantages sont indéniables.

- les fonctionnalités communes à plusieurs classes peuvent être placées dans une superclasse, ce qui permet son utilisation répétée dans toutes les classes situées à un niveau inférieur dans la hiérarchie,
- les modifications apportées à une superclasse sont automatiquement reportées dans toutes ses sous classes, leurs sous-classes ... Nul besoin de charger ou de recompiler une des sous-classes, elles reçoivent par héritage toutes les nouvelles informations.

Avant de coder des classes dans un langage de programmation, il est donc nécessaire de réaliser une étape de **conceptualisation de l'univers à gérer** et de sa **modélisation** à l'aide de diagrammes UML.

6.1 EXEMPLE DE HIERARCHIE DE CLASSES

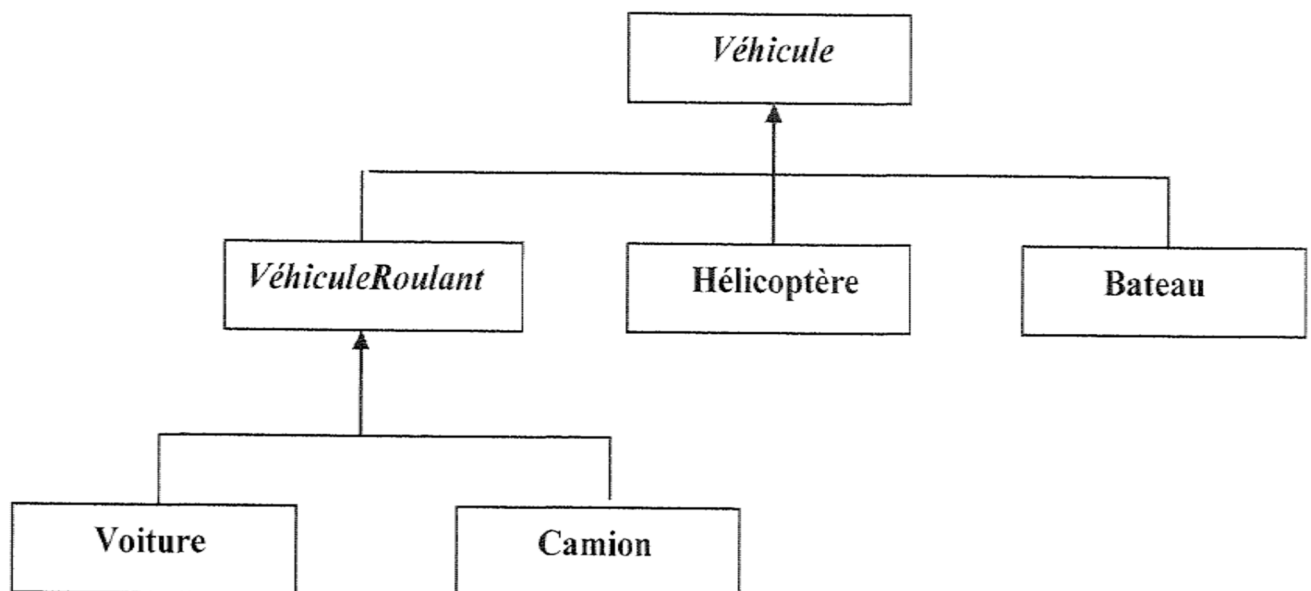
Cet exemple va nous montrer comment utiliser le principe de généralisation / spécialisation pour construire un système orienté objet fiable.

L'entreprise possède un parc de véhicules regroupant :

- des voitures,
- des camions,
- des hélicoptères,
- des bateaux.

Elle souhaite disposer d'un modèle permettant de modéliser le fonctionnement de son parc. Le but est de créer un système de classes permettant de factoriser le plus possible les fonctionnalités de chaque type de véhicule. Partant des catégories à modéliser, il apparaît assez évident de dériver les classes `Voiture` et `Camion` d'une même classe `VéhiculeRoulant` et de laisser de côté les classes `Hélicoptère` et `Bateau`. En outre, bien que différents, tous les véhicules partagent certaines caractéristiques de par leur caractère mobile. Ainsi des actions telles que `Démarrer()`, `Accélérer()`, `Ralentir()` ou `Arrêter()` possèdent un sens pour chaque type de véhicule. Aussi, toutes nos classes vont partager un ancêtre commun que nous appellerons `Véhicule`.

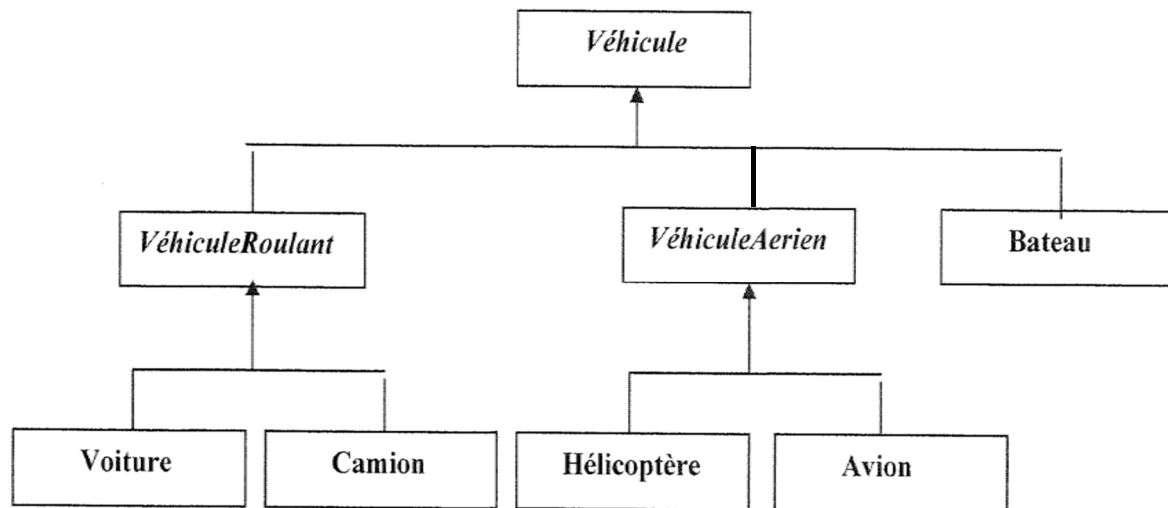
Nous obtenons alors le modèle suivant :



Pour réaliser ce modèle, nous avons procédé par généralisation : à partir de l'ensemble des objets terminaux, nous avons cherché à établir des éléments communs permettant de mettre en évidence des classes de généralisation.

Le procédé de Spécialisation est lui plus utilisé lorsqu'il s'agit de greffer de nouvelles classes dans un système existant. En effet, si la compagnie fait l'acquisition de quelques avions, il sera toujours possible de créer (par Généralisation) une classe `VéhiculeAérien` dont dériveront `Avion` et `Hélicoptère`. On obtient alors le modèle de la figure ci-dessous.

Le principe de généralisation / spécialisation est particulièrement intuitif et puissant car il permet d'identifier des comportements similaires le long d'un arbre de dérivation, chaque sous classe étant à même de choisir entre redéfinir ou hériter du comportement de sa classe mère.



6.2 LES CLASSES ABSTRAITES

En lecteurs attentifs, vous aurez sans doute remarqué que les titres des classes *Véhicule*, *VéhiculeRoulant* et *VéhiculeAerien* sont en italique dans la figure précédente. En sus de rendre le schéma agréable à l'œil, cette présentation n'a rien d'innocent : cela signifie que ces classes sont *abstraites*.

Une classe est dite abstraite ne peut pas être instanciées en l'état et n'est pas forcément complète ; elle existe dans la hiérarchie pour regrouper des attributs et méthodes communs à ce niveau d'abstraction mais certaines méthodes (dites *abstraites*) peuvent ne pas comporter de contenu. Il appartient donc aux classes dérivées de définir du code pour chacune des méthodes abstraites. On parlera alors de classes *concrètes* ; les classes concrètes étant les seules à même d'être instanciées.

Si l'on considère la classe *Véhicule*, il est tout à fait naturel qu'elle ne puisse avoir d'instance : un véhicule ne correspond à aucun objet concret mais plutôt au *concept* d'un objet capable de démarrer, ralentir, accélérer ou s'arrêter, que ce soit une voiture, un camion ou un avion.

7. POUR ALLER PLUS LOIN : LES INTERFACES

Une interface peut être vue comme une **sorte de classe mais vide de contenu** : **une interface ne définit que les signatures des méthodes à implémenter dans les classes** qui se revendiquent de cette interface. Une interface **sert à définir un contrat de service à assurer par une classe**.

Une interface permet à différentes classes qui offrent des services similaires de proposer des méthodes aux signatures identiques. Ainsi, dans le .Net Framework, les différentes classes de collections proposent toutes des méthodes `Add()`, `Remove()` ou `Clear()` car elle implémentant les mêmes interfaces ; l'usage de ces collections est alors simplifié pour le développeur d'application.

Lors de la compilation d'une classe, le compilateur vérifie que toutes les méthodes déclarées dans l'interface sont bien implémentées dans la classe.

Faire la différence entre héritage et utilisation d'interfaces n'est pas toujours aisé et dépend du point de vue du concepteur mais également de l'environnement du logiciel à construire. Une fois de plus se pose le problème de la subjectivité de la décomposition en classes qui dépend fortement de la personnalité du modélisateur et de l'environnement du logiciel à concevoir.

8. POUR CONCLURE SUR L'ORIENTE OBJET

Nous n'avons montré ici que quelques-uns des concepts du modèle orienté objet qui est en réalité beaucoup plus vaste. Le principe d'encapsulation, les relations d'agrégation et d'héritage sont les plus fondamentales.

La structure d'un programme construit selon les principes objet est une collection d'objets communiquant par messages et en interaction constante. Les objets masquent l'organisation du logiciel. Les différents constituants peuvent s'exécuter de façon séquentielle ou concurrente, être situés sur une même machine ou distribués sur un réseau ; le logiciel final est toujours vu du point de vue de l'utilisateur comme un objet monolithique.